COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	July 16, 2026	**DATE SUBMITTED:**	
NAME:	Nichollo Dave A. Galvez	**INSTRUCTOR:**	Engr. Jimlord

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

```cpp
#include <iostream>
#include <string.h>


class Student {
private:
    std::string studentName;
    int studentAge;


public:
    // Constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

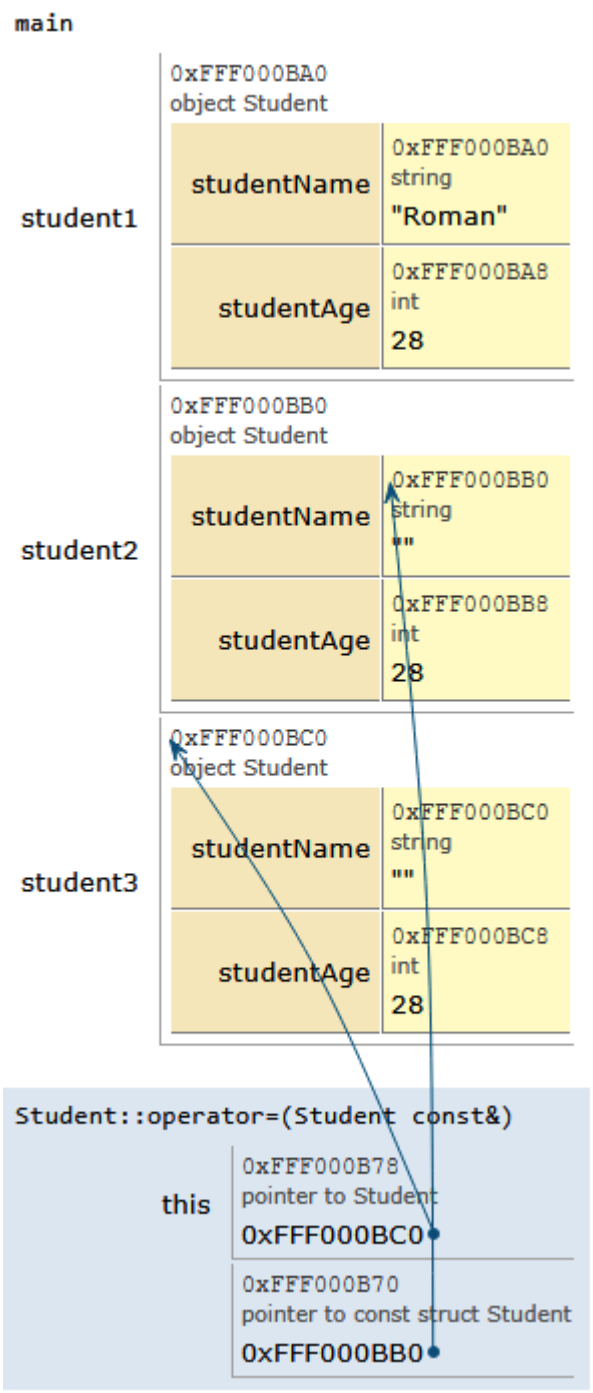

    // Destructor
    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }


    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

```

```
33
34      // Display Attributes
35      void printDetails() {
36          std::cout << this->studentName << " " << this->studentAge << std::endl;
37      }
38  };
39
40
41  int main() {
42      Student student1("Roman", 28);
43      Student student2(student1);
44      Student student3;
45      student3 = student2;
46  return 0;
47  }
```

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```



Observation:

- The code creates three Student objects, where student1 is initialized with custom values, student2(student1) is created using the Copy Constructor, student3 is initialized with default values, and student3 = student2; uses Copy Assignment

before all objects are automatically destroyed by the Destructor at the end of main.

```cpp
#include <iostream>
#include <string.h>


class Student {
private:
    std::string studentName;
    int studentAge;


public:
    // Constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };


    // Destructor
    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

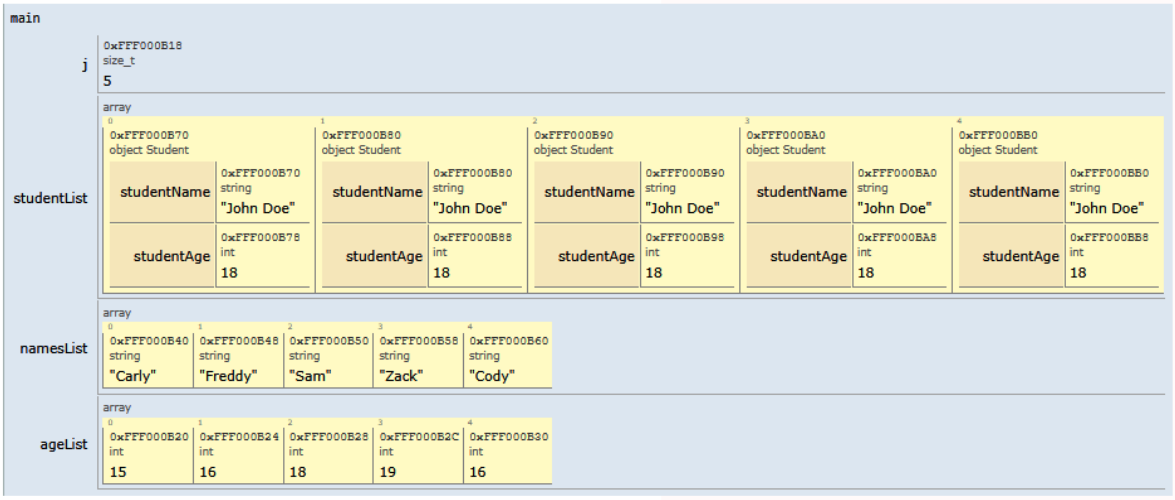

    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

```

```
33
34      // Display Attributes
35      void printDetails() {
36          std::cout << this->studentName << " " << this->studentAge << std::endl;
37      }
38  };
39
40
41  int main() {
42      const size_t j = 5;
43      Student studentList[j] = {};
44      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
45      int ageList[j] = {15, 16, 18, 19, 16};
46      return 0;
47  }
48
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation:



- The code creates an array of 5 Student objects using Student studentList[j] = {};, so the Constructor is called five times with the default values, while the namesList and ageList arrays store sample data that are not yet assigned to the objects before the Destructor is called when the program ends.

```cpp
#include <iostream>
#include <string.h>


class Student {
private:
    std::string studentName;
    int studentAge;


public:
    // Constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };


    // Destructor
    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }


    // Copy Constructor
    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

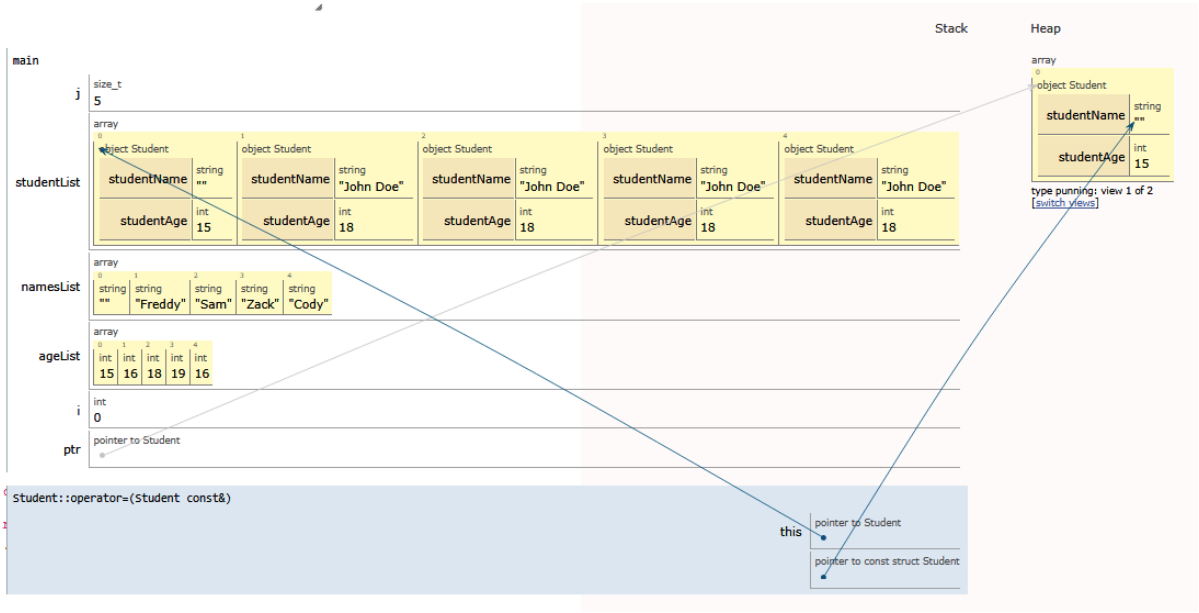
```

```
33
34      // Display Attributes
35      void printDetails() {
36          std::cout << this->studentName << " " << this->studentAge << std::endl;
37      }
38  };
39
40
41  int main() {
42      const size_t j = 5;
43      Student studentList[j] = {};
44      std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
45  int ageList[j] = {15, 16, 18, 19, 16};
46
47  for(int i = 0; i < j; i++){ //loop A
48      Student *ptr = new Student(namesList[i], ageList[i]);
49      studentList[i] = *ptr;
50  }
51  for(int i = 0; i < j; i++){ //loop B
52      studentList[i].printDetails();
53  }
54  return 0;
55  }
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Carly 15
Freddy 16
```

```
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation:



- The code uses Loop A to dynamically create each Student object with new Student(namesList[i], ageList[i]) and copy it into studentList[i], then Loop B calls printDetails() to display the name and age of every student stored in the array.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

```cpp
#include <iostream>
#include <string>

class Grocery {
private:
    std::string name;
    double price;
    int quantity;

public:
    Grocery(std::string n = "", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    Grocery(const Grocery &z) {
        name = z.name;
        price = z.price;
        quantity = z.quantity;
    }

    Grocery& operator=(const Grocery &g) {
        if (this != &g) {
            name = g.name;
            price = g.price;
            quantity = g.quantity;
        }
        return *this;
    }

    virtual ~Grocery() {}

    double calculateSum() {
        return price * quantity;
    }

    virtual void display() {
        std::cout << "Name: " << name << std::endl;
        std::cout << "Total Price: " << price << std::endl;
        std::cout << "Quantity: " << quantity << std::endl;
        std::cout << "Total amount: " << calculateSum() << std::endl;
    }
};

```

```cpp
class Fruit : public Grocery {
public:
    Fruit(std::string ralph, double prc, int q) : Grocery(ralph, prc, q) {}

    void display() override {
        std::cout << "Fruit: " << std::endl;
        Grocery::display();
        std::cout << std::endl;
    }
};

class Vegetable : public Grocery {
public:
    Vegetable(std::string ralph, double prc, int q) : Grocery(ralph, prc, q) {}

    void display() override {
        std::cout << "Vegetable: " << std::endl;
        Grocery::display();
        std::cout << std::endl;
    }
};

double TotalSum(Grocery* list[], int size) {
    double total = 0;

    for (int i = 0; i < size; i++) {
        if (list[i] != nullptr) {
            total += list[i]->calculateSum();
        }
    }

    return total;
}

int main() {
    Grocery* GroceryList[4];

    GroceryList[0] = new Fruit("Watermelon", 20, 17);
    GroceryList[1] = new Fruit("Lychee", 30, 18);
    GroceryList[2] = new Vegetable("Sayote", 60, 12);
    GroceryList[3] = new Vegetable("Carrots", 50, 10);

    std::cout << "Andre's grocery list\n";

    for (int i = 0; i < 4; i++) {
        GroceryList[i]->display();
    }
```

```
93
94      std::cout << "Total amount: " << TotalSum(GroceryList, 4) << std::endl;
95
96      std::cout << "\nDeleting Carrots\n";
97
98      delete GroceryList[3];
99      GroceryList[3] = nullptr;
100
101     std::cout << "\nUpdated Grocery List\n";
102
103     for (int i = 0; i < 4; i++) {
104         if (GroceryList[i] != nullptr) {
105             GroceryList[i]->display();
106         }
107     }
108
109     std::cout << "New Total amount: " << TotalSum(GroceryList, 4) << std::endl;
110
111     for (int i = 0; i < 4; i++) {
112         delete GroceryList[i];
113     }
114
115     return 0;
116 }
```

OUTPUT:

```
Name: Watermelon
Total Price: 20
Quantity: 17
Total amount: 340

Fruit:
Name: Lychee
Total Price: 30
Quantity: 18
Total amount: 540
```

Observations:
The program creates a Grocery pointer array and dynamically stores Fruit and Vegetable objects using constructors. It uses virtual functions and overriding to display each item's details and calculates the total cost through the TotalSum() function.

Summary of lessons learned:
I learned how constructors initialize objects, destructors free memory, and how copy constructors and assignment operators work differently. I also learned how pointers, dynamic memory allocation, inheritance, and virtual functions are used in C++.

Analysis of the procedure:
The activities demonstrated how objects are created, copied, assigned, and destroyed. They showed the use of constructors, copy constructors, assignment operators, arrays of objects, pointers, and dynamic memory through new and delete.

Analysis of the supplementary activity:
The program used inheritance by creating Fruit and Vegetable objects from the Grocery class. It applied dynamic binding through virtual functions to display information and used a function to calculate the total price of all items.

Conclusion / Feedback:
I think I performed well in this activity. I understand the concepts better, but I still need more practice with inheritance, pointers, dynamic memory allocation, and virtual functions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction and explanation of how the following code works (step by step)**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Python Tutor to help organize and explain my ideas more clearly while keeping my original thoughts unchanged. I also used it to assist in finding the necessary formulas for the activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.